



**RAJALAKSHMI
ENGINEERING COLLEGE**

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

CS23432 – Software Construction

(REGULATION 2023)

**RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015**

Name: Prajeet V

Register No: 240701389

Year / Branch / Section: II / CSE

Semester: IV

Academic Year: 2025 - 2026

INDEX

S.No.	Date	Title
1.	07-02-2026	Azure DevOps Environment Setup
2.	14-02-2026	Azure Project Setup & User Stories
3.	21-02-2026	Epics, Features, User Stories Creation
4.	28-02-2026	Sprint Planning using Azure Boards
5.	07-03-2026	Poker Estimation using Azure Boards.
6.	14-03-2026	Designing Class and Sequence Diagrams for Product Category Management System.
7.	04-04-2026	Designing Architectural and ER Diagrams for Product Category Management System.
8.	11-04-2026	Testing – Test Suites and Test Cases in Azure.
9.	18-04-2026	Load Testing and CI/CD Pipeline Setup.
10.	25-04-2026	GitHub: Project Structure & Naming Conventions.

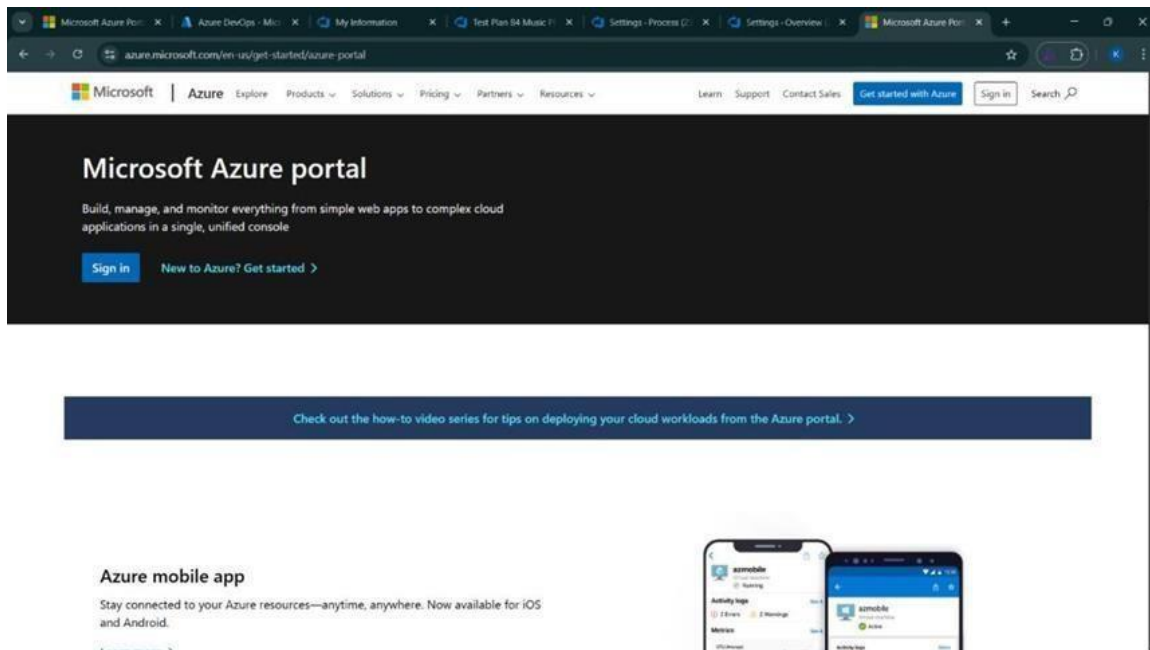
EXP NO: 01	AZURE DEVOPS ENVIRONMENT SETUP
------------	---------------------------------------

Aim:

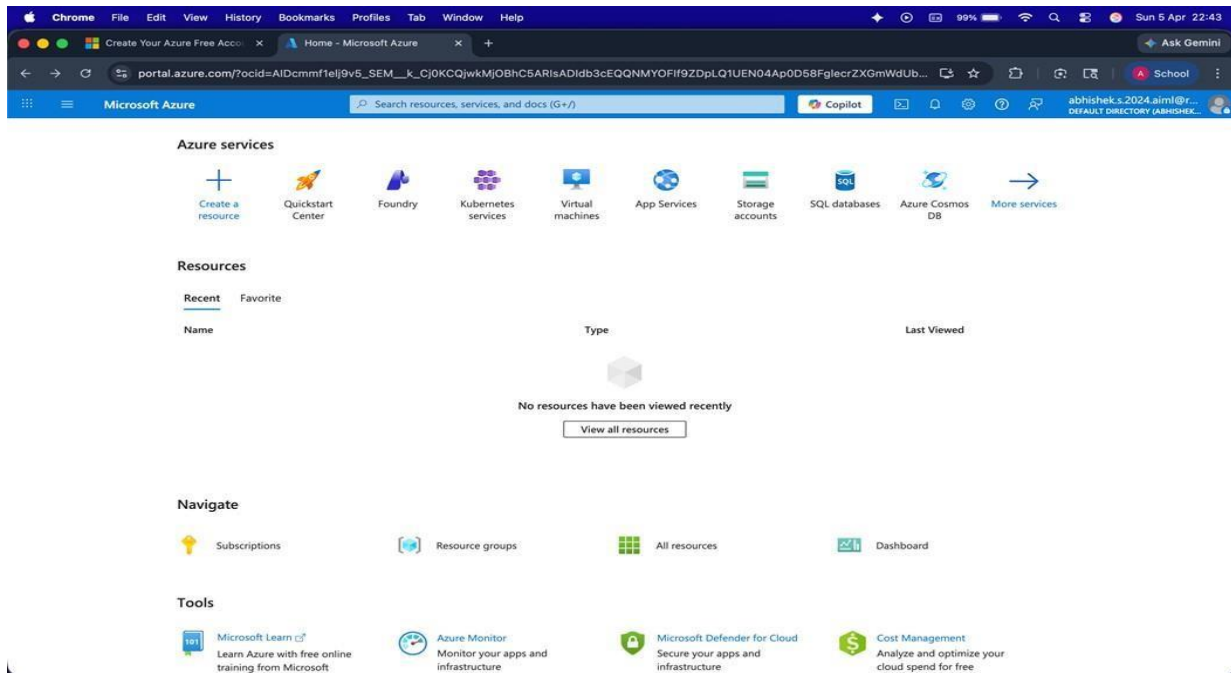
To set up and access the Azure Devops environment by creating an account and workspace for project management.

INSTALLATION

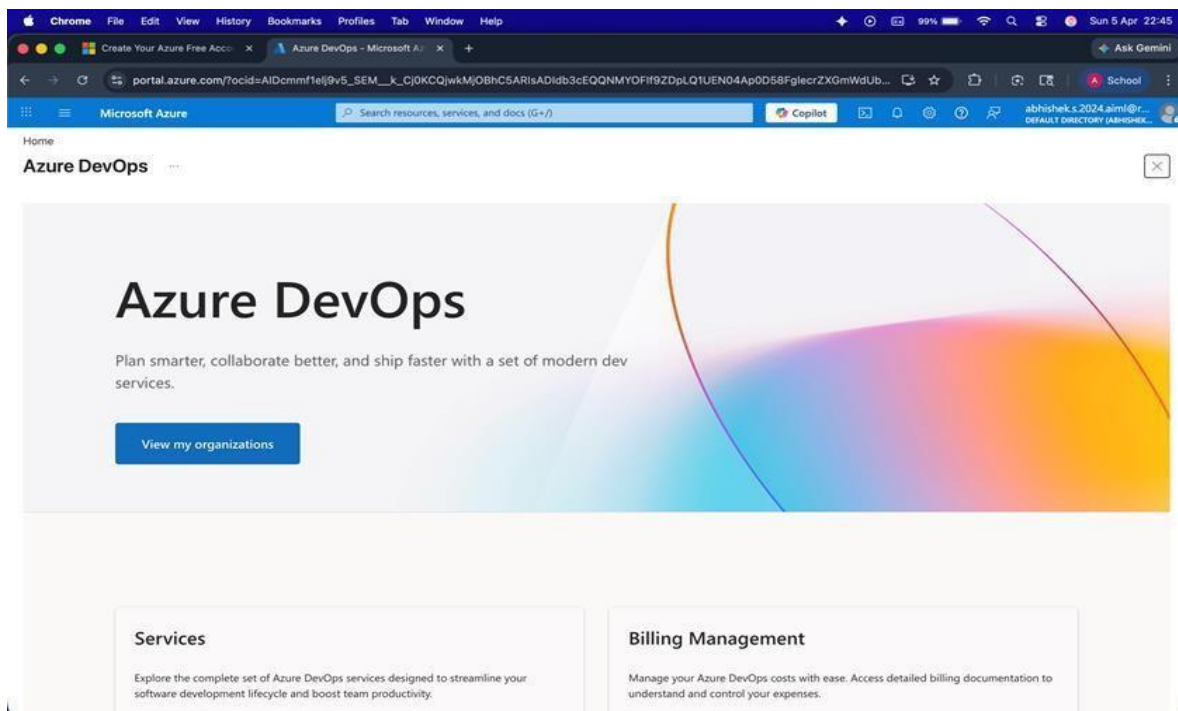
1. Open your web browser and go to: <https://dev.azure.com>
Sign in using your Microsoft account credentials.
If you do not have an account, create one using Sign Up option.



2. Azure DevOps home page



3. Open DevOps environment in the Azure platform by typing **Azure DevOps Organizations** in the search bar.



4. Click on **New Project**, create a new organization/project, and access the Azure DevOps dashboard successfully..

The screenshot shows the Azure DevOps interface for a project named "hospital-appointment-system". The left sidebar contains navigation links: Overview, Summary, Dashboards, Wiki, Boards, Pipelines, and Artifacts. The main content area is divided into two columns. The left column, titled "About this project", contains a project description, objective, key features, and deployment information. The right column, titled "Project stats", shows metrics for Boards, Repos, Pipelines, and Members.

About this project

Project Description: Hospital Appointment System
The Hospital Appointment System is a web-based application designed to simplify and automate the process of booking and managing patient appointments. This system enables users to schedule appointments with doctors efficiently while ensuring scalability, reliability, and performance under real-world traffic conditions.

Objective
The primary goal of this project is to:

- Provide a simple and efficient API for booking hospital appointments
- Handle multiple concurrent users using scalable backend architecture
- Evaluate system performance using modern load testing tools
- Demonstrate cloud deployment and DevOps integration

Key Features

- Appointment Booking API**
Users can book appointments by providing:

Name

Doctor

Date

- High Performance Backend**
Built using Nodejs and Express for fast request handling.
- Database Integration**
Uses MySQL to store appointment data securely.
- Cloud Deployment**
Backend is deployed on Render, making it publicly accessible.

Project stats (Period: Last 7 days)

Boards

- 0 Work items created
- 0 Work items completed

Repos

- 0 Pull requests opened
- 0 Commits by 0 authors

Pipelines

- 0% Builds succeeded

Members 5

Members: JS, PV, GS, PS

Result:

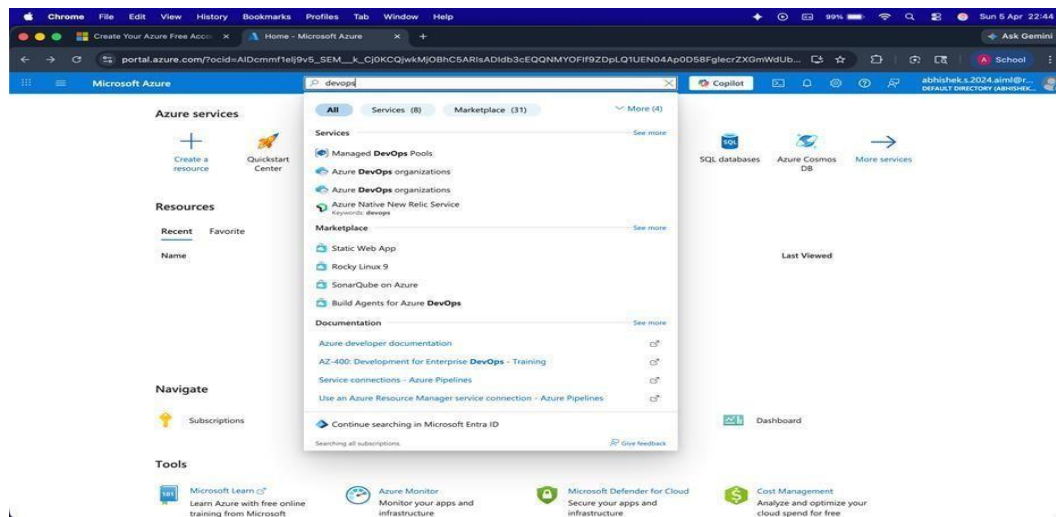
Successfully accessed the Azure environment and created a new workspace/site for project management.

EXP NO: 02	AZURE PROJECT SETUP AND USER STORY MANAGEMENT
------------	--

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

- Create an Azure DevOps Account
- Create the First Project in Your Workspace
 - a. After setup, create your first project.
 - b. Click **New Project**
 - c. Enter details:
 Name: Product Category Management System
 Description: Category handling system
 Template: Scrum
 Access: Private/Open
 - d. Click Create
- Open project dashboard
- Project dashboard
- To manage user stories:
 - a. Go to **Boards** → **Work Items**
 - b. Click **New Work Item** → **User Story**
 - c. Enter details



Result: Successfully created an Azure DevOps project with user story management

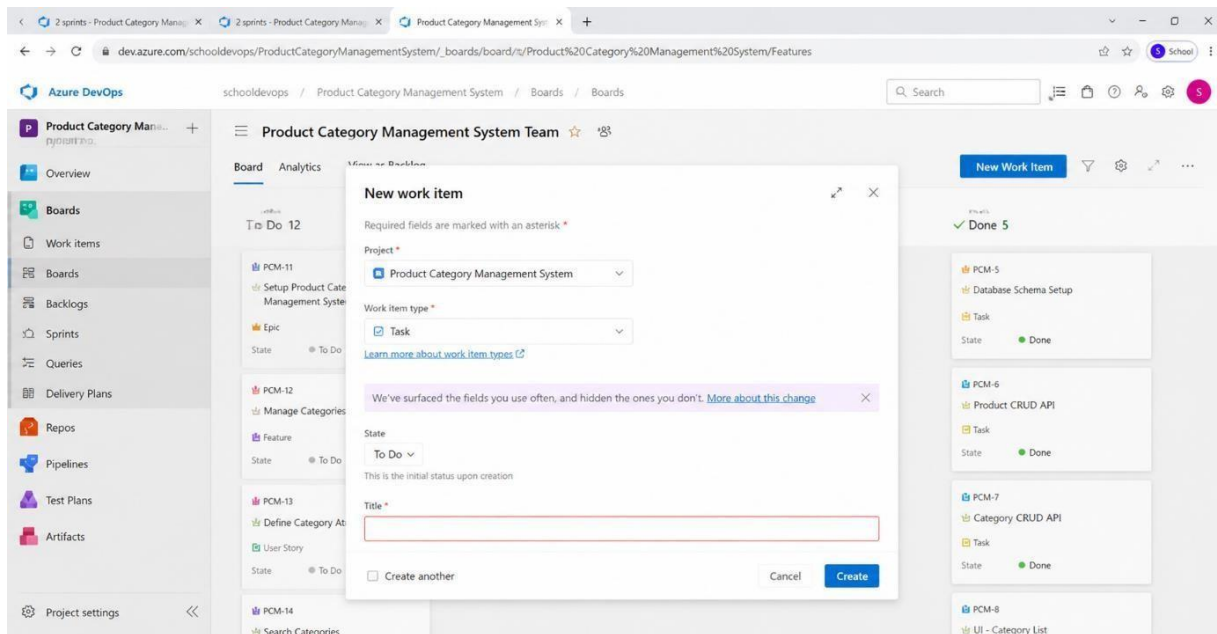
EXP NO: 03

SETTING UP EPICS, TASKS, AND USER STORIES FOR PROJECT PLANNING

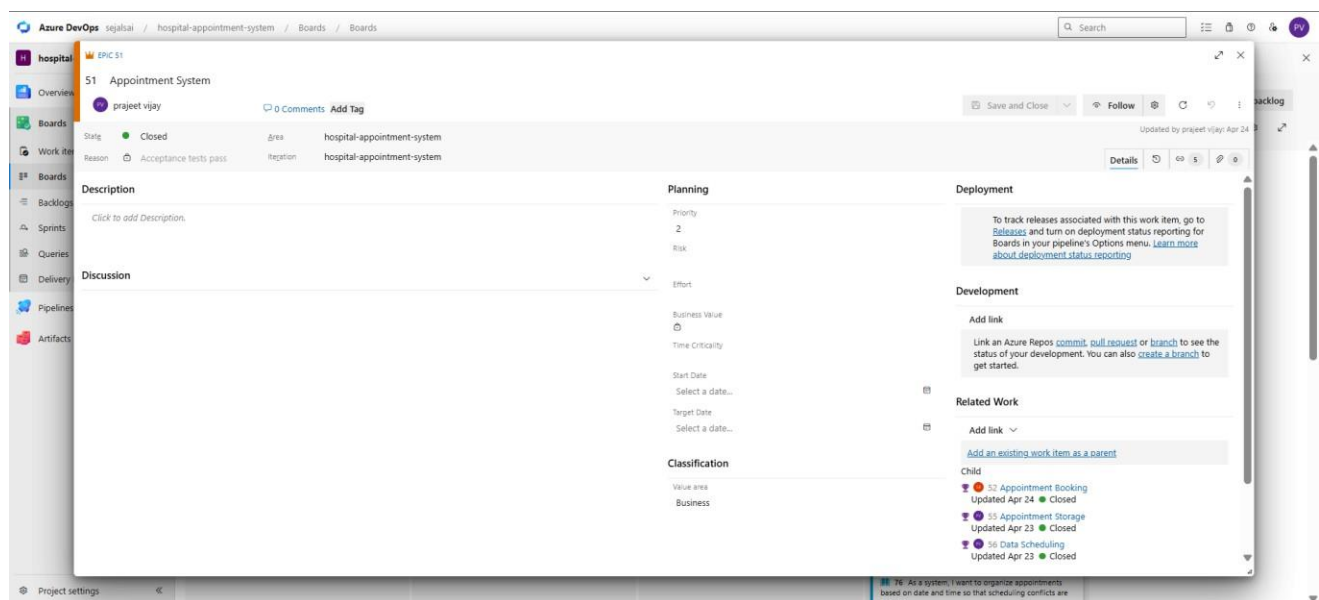
Aim:

To learn about how to create epics, tasks, user stories, and backlogs for your assigned project.

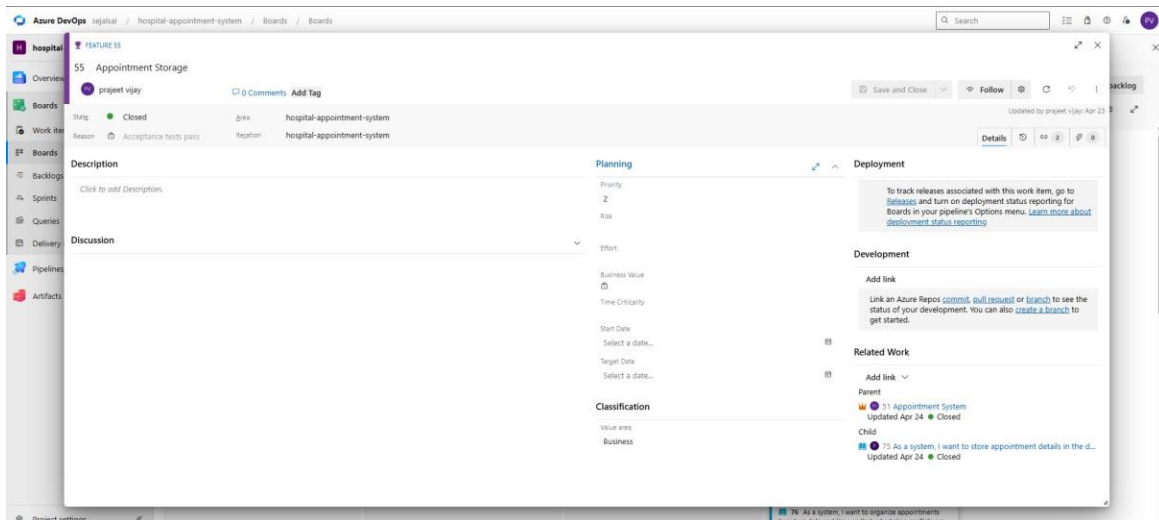
Create Epic, User Stories, Task



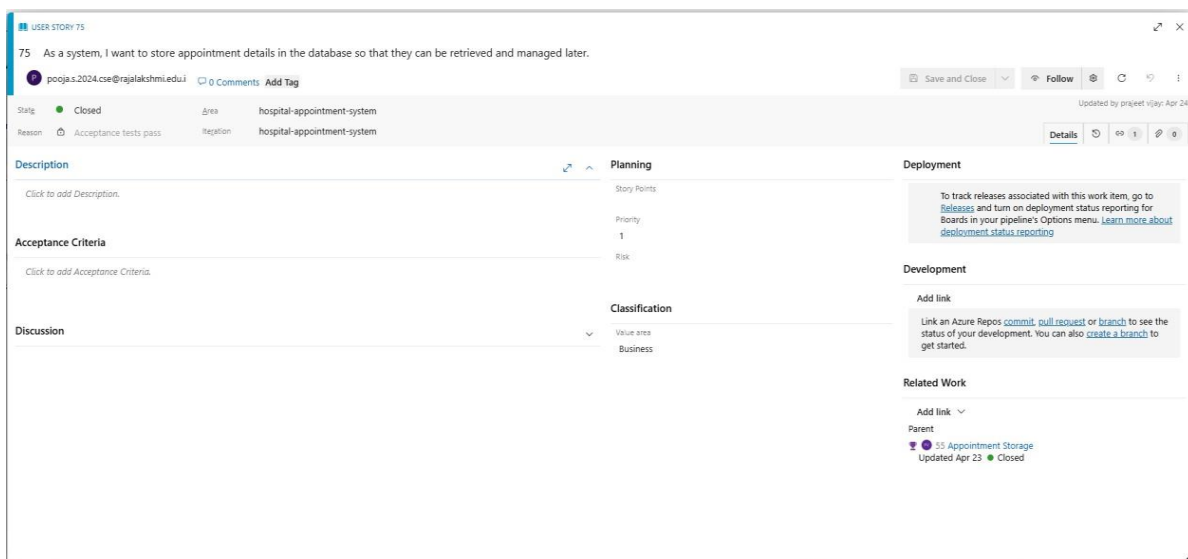
1.Fill in Epics



2.Fill in Features



3.Fill in User Story Details



Result:

Thus, the creation of epics, features, user story and task has been created success

EXP NO: 04

SPRINT PLANNING IN AZURE

Aim:

To assign user story to specific sprint for the Hospital Appointment Management System

Sprint Planning

1. Open **Boards** → **Sprints**
2. Create:
 - Sprint 1
 - Sprint 2
3. Drag user stories into sprint

Sprint 1

The screenshot displays the Azure DevOps Sprints interface for the 'hospital-appointment-system Team'. The left sidebar shows the navigation menu with 'Sprints' selected. The main area shows a Kanban board for 'Sprint 1 - Core Setup - Basic user access simple data display' (March 20 - March 25, 4 work days). The board has columns for 'New', 'Active', 'Resolved', and 'Closed'. A 'Unparented' card is in the 'New' column. The 'Closed' column contains six completed work items, each with a description, status, and assignee.

Work Item ID	Description	Status	Assignee
18	Create appointments table in database	Closed	prajeet vijay
25	Create login page UI using HTML	Closed	Pragith S
26	Create registration page UI using HTML	Closed	sharan bose
27	Create login page UI using HTML	Closed	sharan bose
31	Design input forms for user data	Closed	prajeet vijay
34	Connect frontend to login API	Closed	

Sprint 2

Azure DevOps [sejalrai](#) / [hospital-appointment-system](#) / [Boards](#) / [Sprints](#)

hospital-appointment-... +

Overview

Boards

Work items

Boards

Backlogs

Sprints

Queries

Delivery Plans

Pipelines

Artifacts

Project settings

hospital-appointment-system Team

Taskboard Backlog Analytics

Iteration 2 Person: All

No iteration dates

Collapse all

New Active Resolved Closed

7 User Login and Authentication
Closed
Unassigned

8 Install and configure JWT package
Closed
Unassigned

10 Implement password verification using bcrypt
Closed
Unassigned

11 Protect API routes using authentication middleware
Closed
Unassigned

12 Test login and protected routes using API tester
Closed
Unassigned

13 Update login API to generate JWT token
Closed
Unassigned

15 Create middleware to

Result:

The Sprints are created for the Hospital Appointment Management System

EXP NO: 05	POKER ESTIMATION USING AZURE
-------------------	-------------------------------------

Aim:

Create Poker Estimation for the user stories – Hospital Appointment Management System.

Poker Estimation

Points	Meaning
1	Simple UI text update or small bug fix
2	Login/Register form validation
3	Basic database connection and API setup
5	Appointment booking module with frontend and backend
8	Dashboard with appointment history and management
13	Integration of Azure DevOps, CI/CD pipeline, and database
21	Complete Hospital Management System with all modules and deployment

Procedure:

- Assign points:
 - Add Category → 3
 - Delete Category → 2
 - Search → 1

Sprint	Total SP	Duration	Velocity	Focus
Sprint 1	32 SP	2 weeks	~13SP/dev/sprint	Login, Registration, Database Setup, Basic UI
Sprint 2	48 SP	2 weeks	~31 SP/dev/sprint	Azure Pipelines, Testing, CI/CD Integration

Result:

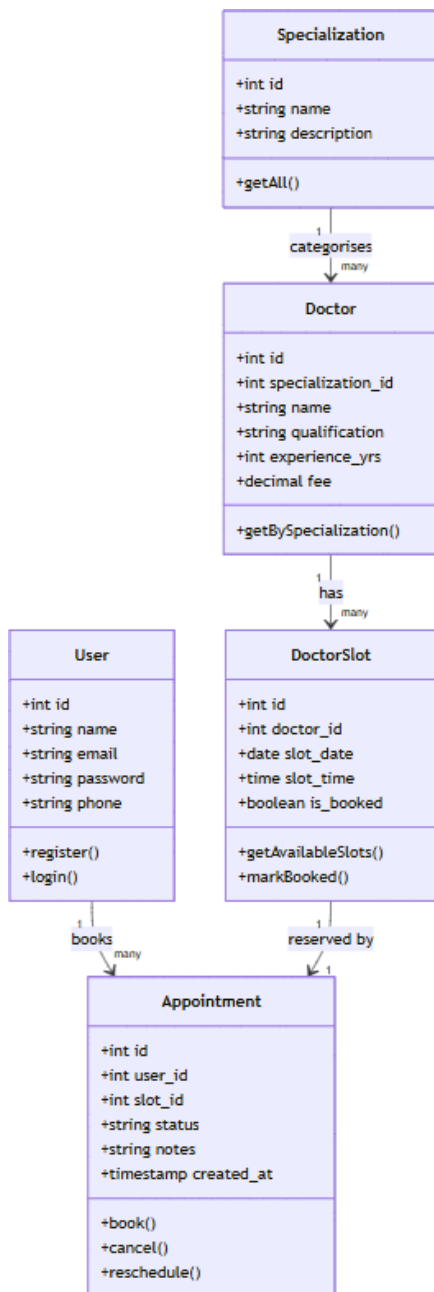
The Estimation/Story Points is created for the project using Poker Estimation.

EXP NO: 06	DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PRODUCT CATEGORY MANAGEMENT SYSTEM
------------	---

Aim:

To Design a Class Diagram and Sequence Diagram for the given Hospital Appointment Management System.

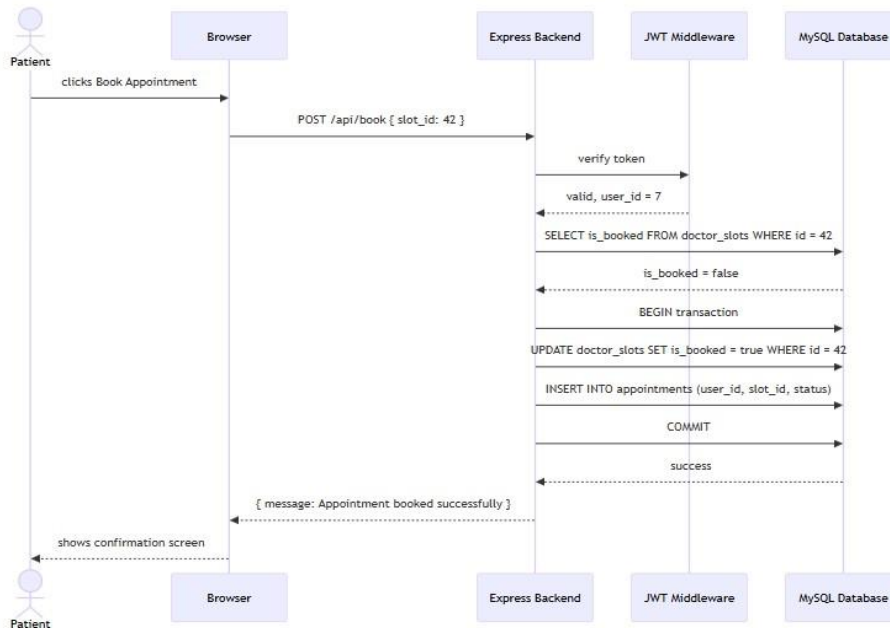
6A. Class Diagram



6B. Sequence Diagram

Sequence Diagram — Appointment Booking Flow

This diagram shows the step-by-step conversation between the browser, backend, authentication middleware, and database when a patient books an appointment.



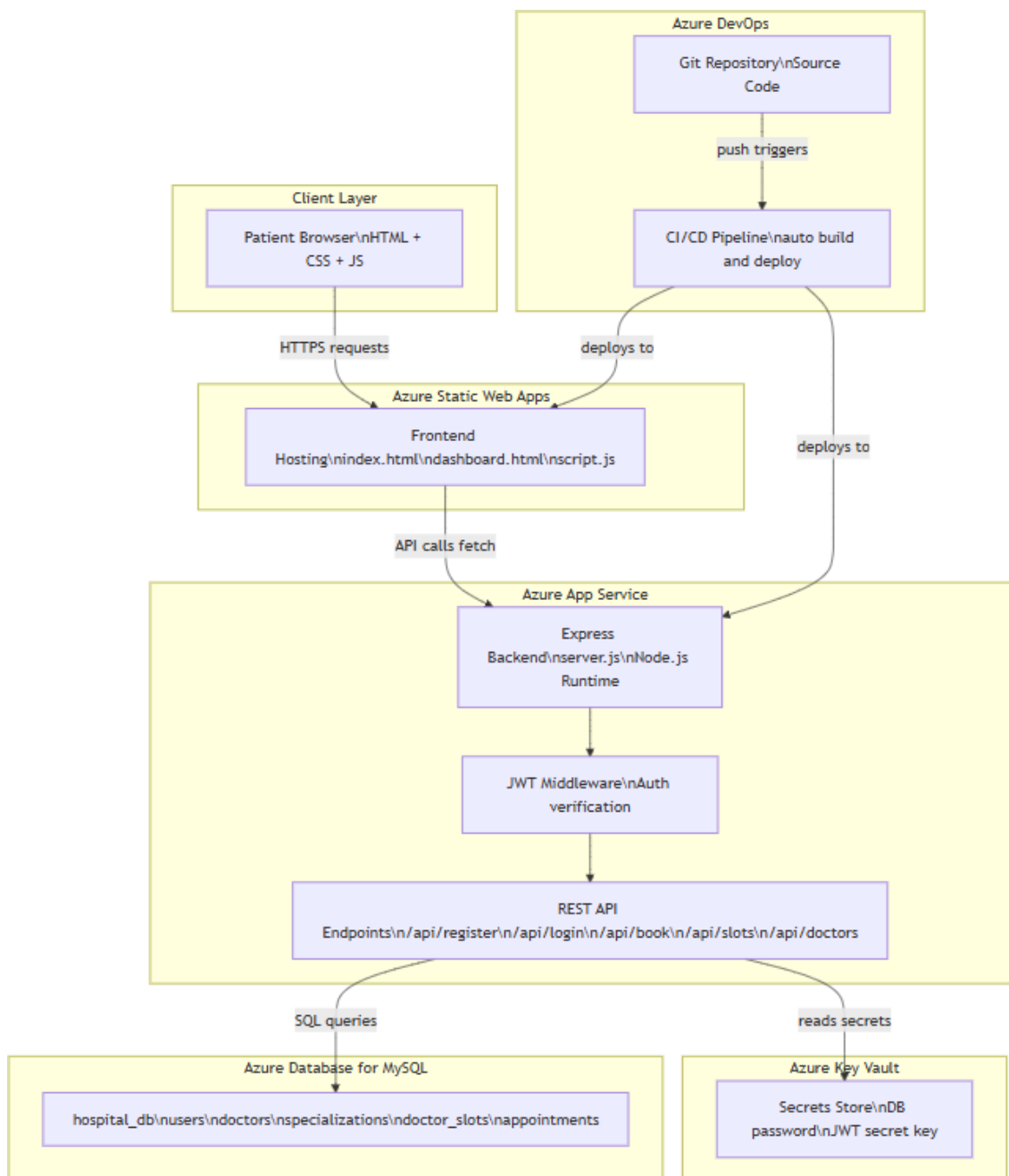
Result: The Class Diagram and Sequence Diagram is designed Successfully for the Hospital Appointment Management System

EXP NO: 07	DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PRODUCT TRACKING SYSTEM
------------	--

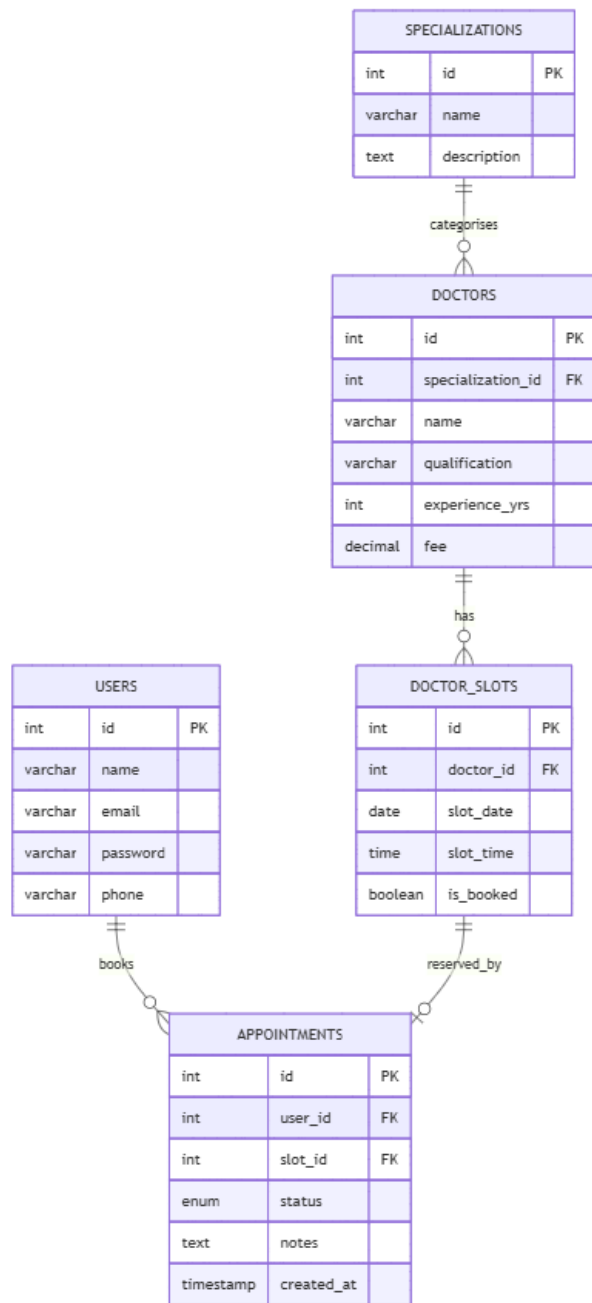
Aim:

To Design an Architectural Diagram and ER Diagram for the given Project.

7 A. Architectural Diagram:



7 B. ER Diagram



Result

The Architecture Diagram and ER Diagram is designed Successfully for the Hospital Appointment Management System.

EXP NO: 08	TESTING – TEST SUITES AND TEST CASES IN AZURE
-------------------	--

Aim:

To create Test Plans and Test Cases for the Hospital Management System and validate core modules such as Login, Registration, Appointment Booking, Dashboard, and Appointment History using Azure DevOps Test Plans.

Test Planning and Test Case Design Procedure:

1. Understand Core Features of the Application

- User Registration
- User Login
- Dashboard Navigation
- Appointment Booking
- View Appointment History

2. Define User Interactions

- Each test case simulates real hospital management workflows.
- Both valid and invalid scenarios are tested.

3. Design Happy Path Test Cases

- Verify successful registration and login.
- Verify appointment booking flow.
- Verify dashboard accessibility.

4. Design Error Path Test Cases

- Invalid login credentials.
- Empty registration fields.
- Booking without selecting doctor/date.
- Invalid appointment inputs.

5. Break Down Steps and Expected Results

- Each test contains:
- User actions
- Input data
- Expected output
- Validation message

6. Use Clear Naming and IDs

- TC01 – Successful Login
- TC02 – Invalid Login
- TC03 – Successful Registration
- TC04 – Empty Registration
- TC05 – Appointment Booking

7. Separate Test Suites

- Login Management
- Registration Management
- Appointment Management
- Dashboard & History Validation

8. Prioritize and Review

- Core hospital workflows marked as High Priority.
- Test cases reviewed for completeness and traceability.

USER STORIES

- As a user, I want to register into the system securely so that I can access hospital services. (US01)
- As a user, I want to log in securely using my email and password. (US02)
- As a patient, I want to book appointments with doctors easily. (US03)
- As a user, I want to view my appointment details and booking history. (US04)
- As an admin/system, I want all appointment records stored in the database. (US05)
-

Test Suites

Test Suite: TS01 – User Registration

TC01 – Successful Registration

Action:

- Open Register page.
- Enter valid Name, Email, Password, and Phone Number.
- Click “Register”.

Expected Results:

- Registration successful message displayed.
- User details stored in MySQL database.
- Redirected to Login page.

Type:

Happy Path

TC02 – Empty Registration Fields

Action:

- Open Register page.
- Leave fields empty.
- Click “Register”.

Expected Results:

- Validation/error message displayed.
- Registration prevented.

Type:

Error Path

Test Suite: TS02 – User Login**TC03 – Successful Login****Action:**

- Open Login page.
- Enter valid Email and Password.
- Click “Login”.

Expected Results:

- Login successful message displayed.
- Redirected to Dashboard page.

Type:

Happy Path

TC04 – Invalid Login**Action:**

- Enter invalid email/password.
- Click “Login”.

Expected Results:

- “Invalid Email or Password” message displayed.

Type:

Error Path

Test Suite: TS03 – Appointment Booking**TC05 – Successful Appointment Booking****Action:**

- Login successfully.
- Open Booking page.
- Select doctor and date.
- Click “Book”.

Expected Results:

- Appointment booked successfully.
- Data inserted into appointments table.
- Redirected to View page.

Type:

Happy Path

TC06 – Booking Without Doctor Selection**Action:**

- Open Booking page.
- Leave doctor unselected.
- Click “Book”.

Expected Results:

- Validation message displayed.
- Booking prevented.

Type:

Error Path

Test Suite: TS04 – Dashboard and Appointment History

TC07 – Dashboard Access

Action:

- Login successfully.
- Navigate to Dashboard.

Expected Results:

- Dashboard displayed successfully.
- User options visible.

Type:

Happy Path

TC08 – View Appointment History

Action:

- Open View Appointments page.

Expected Results:

- Appointment records fetched from database.
- Appointment details displayed correctly.

Type:

Happy Path

TEST CASES:

70	TC07 – View Appointments	prajeet vijay	Design	hospital-appointment-system	4/23/2026 9:04:47 AM
69	TC06 – Booking (Empty Fields)	prajeet vijay	Design	hospital-appointment-system	4/23/2026 9:03:29 AM
68	TC05 – Appointment Booking (Valid)	prajeet vijay	Design	hospital-appointment-system	4/23/2026 9:02:06 AM
67	TC04 – Login (Invalid Password)	prajeet vijay	Design	hospital-appointment-system	4/23/2026 8:59:29 AM
66	TC03 – Login (Valid Credentials)	prajeet vijay	Design	hospital-appointment-system	4/23/2026 8:55:15 AM
65	TC02 – Registration (Duplicate Email)	prajeet vijay	Design	hospital-appointment-system	4/23/2026 8:53:45 AM
64	TC01 – Verify User Registration with valid data	prajeet vijay	Design	hospital-appointment-system	4/23/2026 8:48:39 AM

TEST CASE: 70			
70	TC07 – View Appointments	prajeet vijay	0 Comments Add Tag
Story	Design	Dev	hospital-appointment-system
Reason	New	Regression	hospital-appointment-system
Steps			
Steps Action Expected result Attachments			
1.	Open view page	Page loads	
2.	System fetches data	Data retrieved	
3.	Display appointments	List/table shown	
Click on type here to add a step			

RESULT:

Successfully created and executed Test Plans and Test Cases for the Hospital Management System in Azure DevOps. Both happy path and error path scenarios were validated for Registration, Login, Appointment Booking, Dashboard, and Appointment History modules

EXP NO: 09	LOAD TESTING AND CI/CD PIPELINE SETUP
-------------------	--

AIM:

Load Testing is used to simulate multiple users accessing the Hospital Management System simultaneously to evaluate application performance, response time, stability, and scalability under heavy traffic conditions. Azure Load Testing was used to monitor the application's behavior and identify performance bottlenecks before deployment.

Steps to Perform Load Testing Using Azure

1. Open Azure Portal

- Sign in to Microsoft Azure Portal.
- Open Azure Load Testing service.

2. Create Load Testing Resource

- Click:

Create Resource

- Enter:
 - Resource Name
 - Subscription
 - Resource Group
 - Region

3. Create a New Test

- Open Azure Load Testing dashboard.
- Click:

Create Test

4. Configure Test Settings

Parameter	Value
Test Type	URL Based Test
Users	50 Virtual Users
Duration	2 Minutes
Ramp-Up Time 30 Seconds	

4. Configure Endpoint

Field	Value
Server URL	http://localhost:3000
API Path	/login
Method	POST

6. Add Request Body

```
{  
  "email": "test@mail.com",  
  "password": "123456"  
}
```

7. Run Load Test

- Save the configuration.
- Click:

Run Test

8. Monitor Performance Metrics

Observe:

- Response Time
- Requests per Second
- Throughput
- Error Percentage
- CPU and Memory Usage

Result:

Successfully performed Load Testing on the Hospital Management System using Azure Load Testing. Multiple virtual users accessed the Login API simultaneously, and system performance metrics such as response time, throughput, and server stability were analyzed successfully.

CI/CD Pipeline

Steps to Create and Demonstrate CI/CD Pipeline

1. Open GitHub Repository

Open the Hospital Management System repository in GitHub.

Repository:

[Hospital-Management-System Repo](#)

2. Open GitHub Actions

- Navigate to:

Actions Tab

- Create a new workflow.

3. Create Workflow File

Create:

`.github/workflows/nodejs.yml`

4. Configure Pipeline Stages

The pipeline performs:

- Install dependencies
- Build Node.js application
- Run test cases
- Validate project files
- Trigger deployment workflow

5. Save Workflow File

Commit and push workflow configuration to GitHub.

6. Trigger Pipeline

- Push code changes to repository.
- GitHub Actions automatically starts CI/CD workflow.

7. Verify Pipeline Execution

Check:

- Build Status
- Test Execution
- Deployment Logs
- Workflow Success Status

CI/CD Workflow Used

YAML

name: Hospital Management CI/CD

on:

push:

branches:

- main

jobs:

build:

runs-on: ubuntu-latest

steps:

- **name:** Checkout Code

uses: actions/checkout@v3

- **name:** Setup Node.js

uses: actions/setup-node@v3

with:

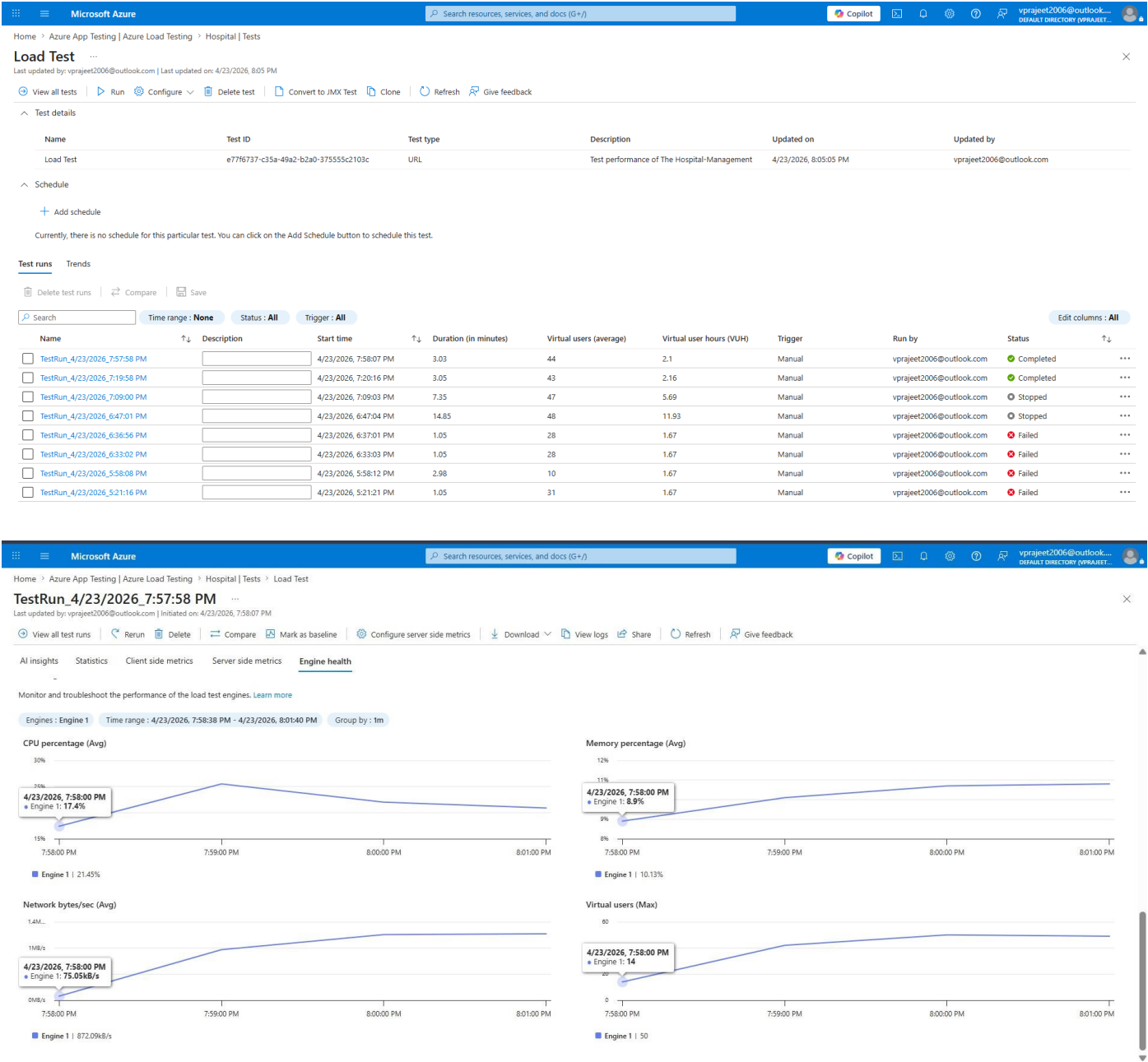
node-version: 18

- **name:** Install Dependencies

run: npm install

- name: Run Tests
run: echo "Tests Executed Successfully"

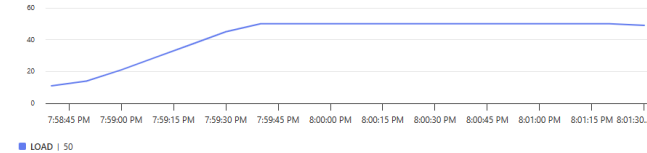
- name: Build Application
run: echo "Build Successful"



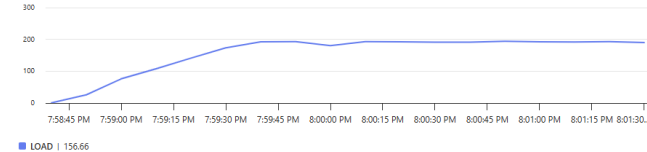
Client-side metrics

Requests : LOAD Aggregation : P90 Error Type : 502 Time range : 4/23/2026, 7:58:38 PM - 4/23/2026, 8:01:40 PM Group by : 10s

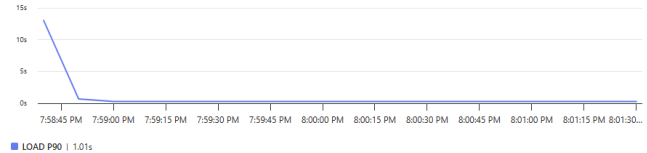
Virtual Users (Max)



Requests/sec (Avg)



Response time (successful responses)



Errors (total)



Home > Azure App Testing | Azure Load Testing > Hospital | Tests > Load Test

TestRun_4/23/2026_7:57:58 PM

Last updated by: vprajeet2006@outlook.com | Initiated on: 4/23/2026, 7:58:07 PM

View all test runs Rerun Delete Compare Mark as baseline Configure server side metrics Download View logs Share Refresh Give feedback

Test run feedback How was your experience running a load test with Azure Load Testing?

Completed

Run completed No test criteria

Start time : 4/23/2026, 7:58:38 PM

End time : 4/23/2026, 8:01:40 PM

Duration : 3 mins, 2 secs

Engine instances : 1

Virtual users (Average) : 44

Virtual user hours : 2.1

Test run ID : aa9c38c6-df34-4355-b343-a832f218202f

Baseline : Set baseline

AI insights Statistics Client side metrics Server side metrics Engine health

AI summary and insights : Click to discover AI-powered insights that help debug and improve your app's performance. Learn more Get insights

Server-side metrics aren't configured. Configure them now to unlock deeper AI-based insights and results. Learn more

Statistics

Load

28199

Total requests

Duration

3 mins, 2 secs

Response time

276.00 ms

90th percentile response time

Error percentage

0 %

Aggregate requests which failed

Throughput

157.54 /s

Request rate

> Request statistics

Client-side metrics

Requests : LOAD Aggregation : P90 Error Type : 502 Time range : 4/23/2026, 7:58:38 PM - 4/23/2026, 8:01:40 PM Group by : 10s

Virtual Users (Max)

Response time (successful responses)

Recent AI Runs

All pipeline runs		Steps
Description		
Create azure-webapps-node.yml	✔ P300051511 on Project302550Credential CI for 37 min 46 sec 47s	✔
Update azure-pipelines.yml for Azure Pipelines	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Update azure-pipelines.yml for Azure Pipelines	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Update azure-pipelines.yml for Azure Pipelines	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Update azure-pipelines.yml for Azure Pipelines	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Update azure-pipelines.yml for Azure Pipelines	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Update azure-pipelines.yml for Azure Pipelines	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Update azure-pipelines.yml for Azure Pipelines	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Fix for render - azure	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Merge branch 'main' of https://github.com/Project302550	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Update azure-pipelines.yml	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Update azure-pipelines.yml	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔
Set up CI with Azure Pipelines	✔ P300042476 on Project302550Manually run 37 min 46 sec 47s	✔

#20260429.6 • Update azure-pipelines.yml for Azure Pipelines

▲ Pipeline2025.SC

🕒 This run is being retained at one of 3 recent runs by pipeline.

Summary Code Coverage

Manually run by  prajet vijay

🕒 Apr 29 at 10:16 PM 🕒 47s

Repository and version

📁 Pipeline2025.SC

📁 main 📁 64710d2

Related

📁 2 work items

📁 0 published

Tests and coverage

📁 0 Get started

Warnings

🚨 Task 'Node.js tool installer' version 0 (NodeTool@0) is deprecated

📄 NodeTool@0

🚨 This task is deprecated and will no longer receive updates. Please use UseNodeV1 as a replacement.

📄 UseNodeV1

Stages jobs

🟢 Stage

1 job completed 40%

🟢 job 40%

🔄 Rerun Stage

Initialize job

```
1 Starting: Initialize job
2 Agent name: 'LATITUDE5520'
3 Agent machine name: 'LATITUDE5520'
4 Current agent version: '4.272.0'
5 Agent running as: 'User'
6 Prepare build directory.
7 Set build variables.
8 Download all required tasks.
9 ##[warning]Task 'Node.js tool installer' version 0 (NodeTool@0) is deprecated.
10 ##[warning]This task is deprecated and will no longer receive updates. Please use UseNodeV1 as a replacement.
11 Checking job knob settings.
12 Knob: DockerActionRetries = true Source: $(VSTSAGENT_DOCKER_ACTION_RETRIES)
13 Knob: UseGitLongPaths = true Source: $(USE_GIT_LONG_PATHS)
14 Knob: UseNode24withHandlerData = True Source: $(DistributedTask.Agent.UseNode24withHandlerData)
15 Knob: EnableIssueSourceValidation = true Source: $(ENABLE_ISSUE_SOURCE_VALIDATION)
16 Knob: AgentEnablePipelineArtifactLargeChunkSize = true Source: $(AGENT_ENABLE_PIPELINEARTIFACT_LARGE_CHUNK_SIZE)
17 Knob: ContinueAfterCancelProcessTreeKillAttempt = true Source: $(VSTSAGENT_CONTINUE_AFTER_CANCEL_PROCESSTREEKILL_ATTEMPT)
18 Knob: ProcessHandlerSecureArguments = false Source: $(AZP_75787_ENABLE_NEW_LOGIC)
19 Knob: ProcessHandlerSecureArguments = false Source: $(AZP_75787_ENABLE_NEW_LOGIC_LOG)
20 Knob: ProcessHandlerTelemetry = true Source: $(AZP_75787_ENABLE_COLLECT)
21 Knob: UseNewNodeHandlerTelemetry = True Source: $(DistributedTask.Agent.USENEWNODEHANDLERTELEMETRY)
22 Knob: ProcessHandlerEnableNewLogic = true Source: $(AZP_75787_ENABLE_NEW_PH_LOGIC)
23 Knob: EnableResourceMonitorDebugOutput = true Source: $(AZP_ENABLE_RESOURCE_MONITOR_DEBUG_OUTPUT)
24 Knob: EnableResourceUtilizationWarnings = true Source: $(AZP_ENABLE_RESOURCE_UTILIZATION_WARNINGS)
25 Knob: IgnoreVSTaskLib = true Source: $(AZP_AGENT_IGNORE_VSTASKLIB)
26 Knob: FailJobWhenAgentDies = true Source: $(FAIL_JOB_WHEN_AGENT_DIES)
27 Knob: EnhancedWorkerCrashHandling = true Source: $(AZP_ENHANCED_WORKER_CRASH_HANDLING)
28 Knob: CheckForTaskDeprecation = true Source: $(AZP_AGENT_CHECK_FOR_TASK_DEPRECATION)
29 Knob: CheckIfTaskNodeRunnerIsDeprecated246 = True Source: $(DistributedTask.Agent.CheckIfTaskNodeRunnerIsDeprecated246)
30 Knob: EnableTimeoutLogFlushing = True Source: $(DistributedTask.Agent.EnableTimeoutLogFlushing)
31 Knob: LogTaskNameInUserAgent = true Source: $(AZP_AGENT_LOG_TASKNAME_IN_USERAGENT)
32 Knob: UseFetchFilterInCheckoutTask = true Source: $(AGENT_USE_FETCH_FILTER_IN_CHECKOUT_TASK)
33 Knob: Rosetta2Warning = true Source: $(ROSETTA2_WARNING)
34 Knob: CheckPsModulesLocations = False Source: $(DistributedTask.Agent.CheckPsModulesLocations)
35 Knob: AddForceCredentialsToGitCheckoutEnhanced = True Source: $(DistributedTask.Agent.AddForceCredentialsToGitCheckoutEnhanced)
36 Knob: UseLatestTFExe = True Source: $(DistributedTask.Agent.UseLatestTFExe)
37 Knob: UseSparseCheckoutInCheckoutTask = true Source: $(AGENT_USE_SPARSE_CHECKOUT_IN_CHECKOUT_TASK)
38 Knob: AgentCDNConnectivityFailWarning = False Source: $(DistributedTask.Agent.AgentCDNConnectivityFailWarning)
39 Knob: UseEnhancedLogging = true Source: $(AZP_USE_ENHANCED_LOGGING)
40 Finished checking job knob settings.
41 Start tracking orphan processes.
42 Finishing: Initialize job
```

Summary

All jobs

build

Run details

Usage

Workflow file

build

Started 15s ago

Search logs

Upload artifact for deployment job

3s

22 Artifact node-app has been successfully uploaded: Final Size is 3414174 bytes. Artifact ID is 7017017131

23 Artifact download URL: <https://github.com/Prajeet2625/SC/actions/runs/25919640917/artifacts/7017017131>

Post Set up Node.js

0s

1 Post job cleanup.

2 Cache hit occurred on the primary key node-cache-Linux-x64-npm-a804c543c39973a49906ee4eca0cfac8b47be290f24cfe0e3cff69419522034, not saving cache.

Post Run actions/checkout@v4

1s

1 Post job cleanup.

2 /usr/bin/git version

3 git version 2.54.0

4 Temporarily overriding HOME='/home/runner/work/_temp/2e39afcc-c960-41bd-8045-824cbff52b20' before making global git config changes

5 Adding repository directory to the temporary git global config as a safe directory

6 /usr/bin/git config --global --add safe.directory /home/runner/work/SC/SC

7 /usr/bin/git config --local --name-only --get-regexp core.sshCommand

8 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'core.sshCommand' && git config --local --unset-all 'core.sshCommand' || :"

9 /usr/bin/git config --local --name-only --get-regexp http.https://github.com/.extraheader

10 http.https://github.com/.extraheader

11 /usr/bin/git config --local --unset-all http.https://github.com/.extraheader

12 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'http.https://github.com/.extraheader' && git config --local --unset-all 'http.https://github.com/.extraheader' || :"

13 /usr/bin/git config --local --name-only --get-regexp *includeIf.gitdir:

14 /usr/bin/git submodule foreach --recursive git config --local --show-origin --name-only --get-regexp remote.origin.url

Result:

Successfully implemented and demonstrated a CI/CD pipeline for the Hospital Management System using GitHub Actions. The workflow automatically builds and validates the application whenever code changes are pushed to the repository.

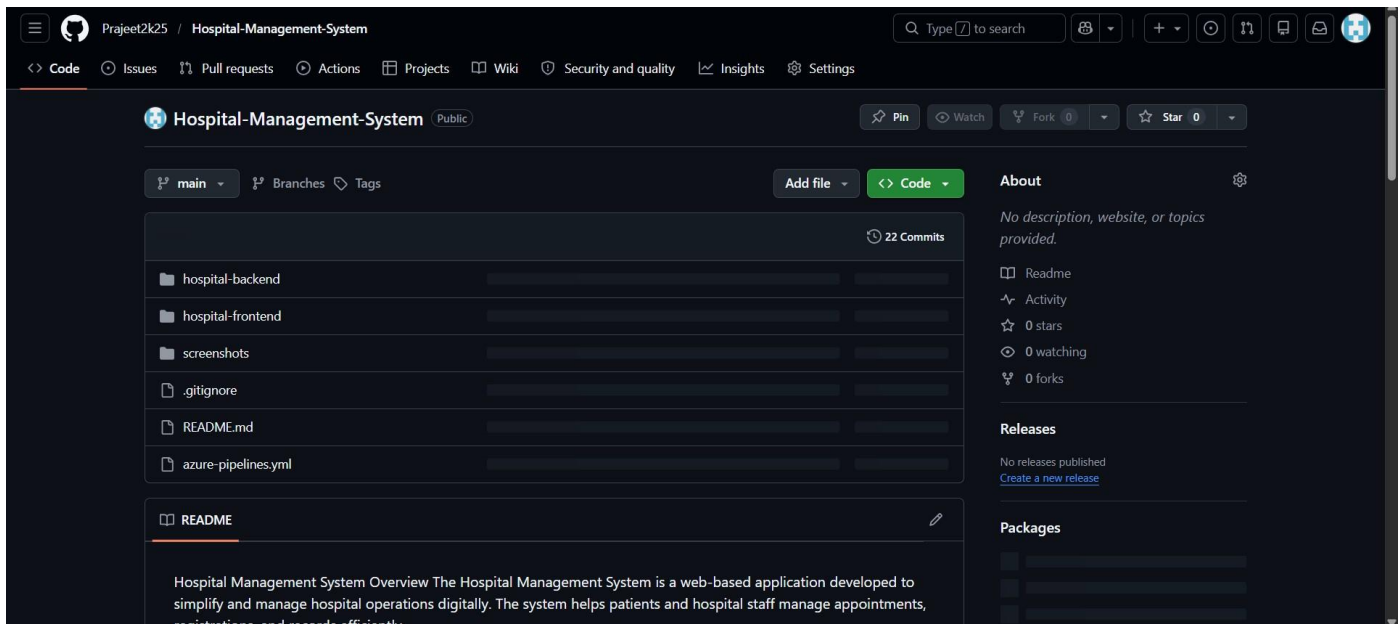
EXP NO: 10

GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Product Tracking System project.

GitHub Project Structure



Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the code.